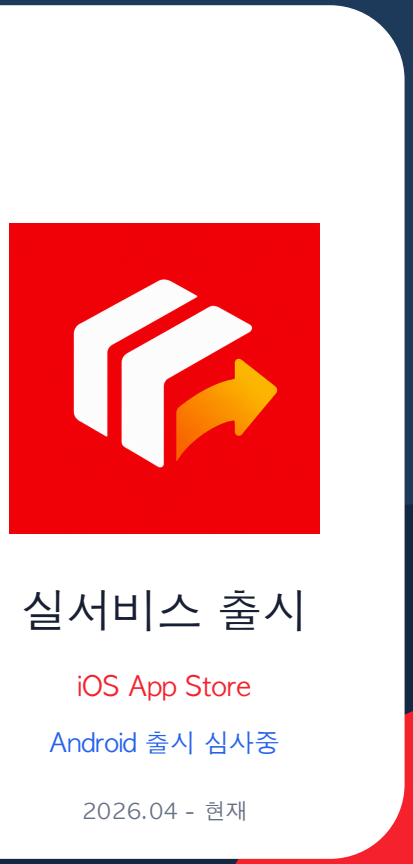
FULL-STACK DEVELOPMENT PORTFOLIO

주문왔슈

소상공인 주문·택배 업무 통합 플랫폼

문자 주소 요청부터 주문 등록, 택배 접수, 송장 출력, 배송 상태 확인까지
웹·모바일·백엔드·배포를 하나의 흐름으로 연결했습니다.



React · React Native · Spring Boot · Native Printer · Docker

구매자 웹 · 판매자 앱 · 운영자 백오피스 · 도메인 API · 배포 운영

전화 주문의 반복 업무를 하나의 흐름으로

고객 주소를 다시 받아 옮겨 적던 과정을 모바일 링크와 앱 중심의 업무 흐름으로 바꿨습니다.



BEFORE

주소 메모와 재입력

- 고객 주소를 전화나 문자로 다시 확인
- 판매자와 택배기사가 정보를 반복 입력
- 주문·접수·송장·배송 상태가 분리

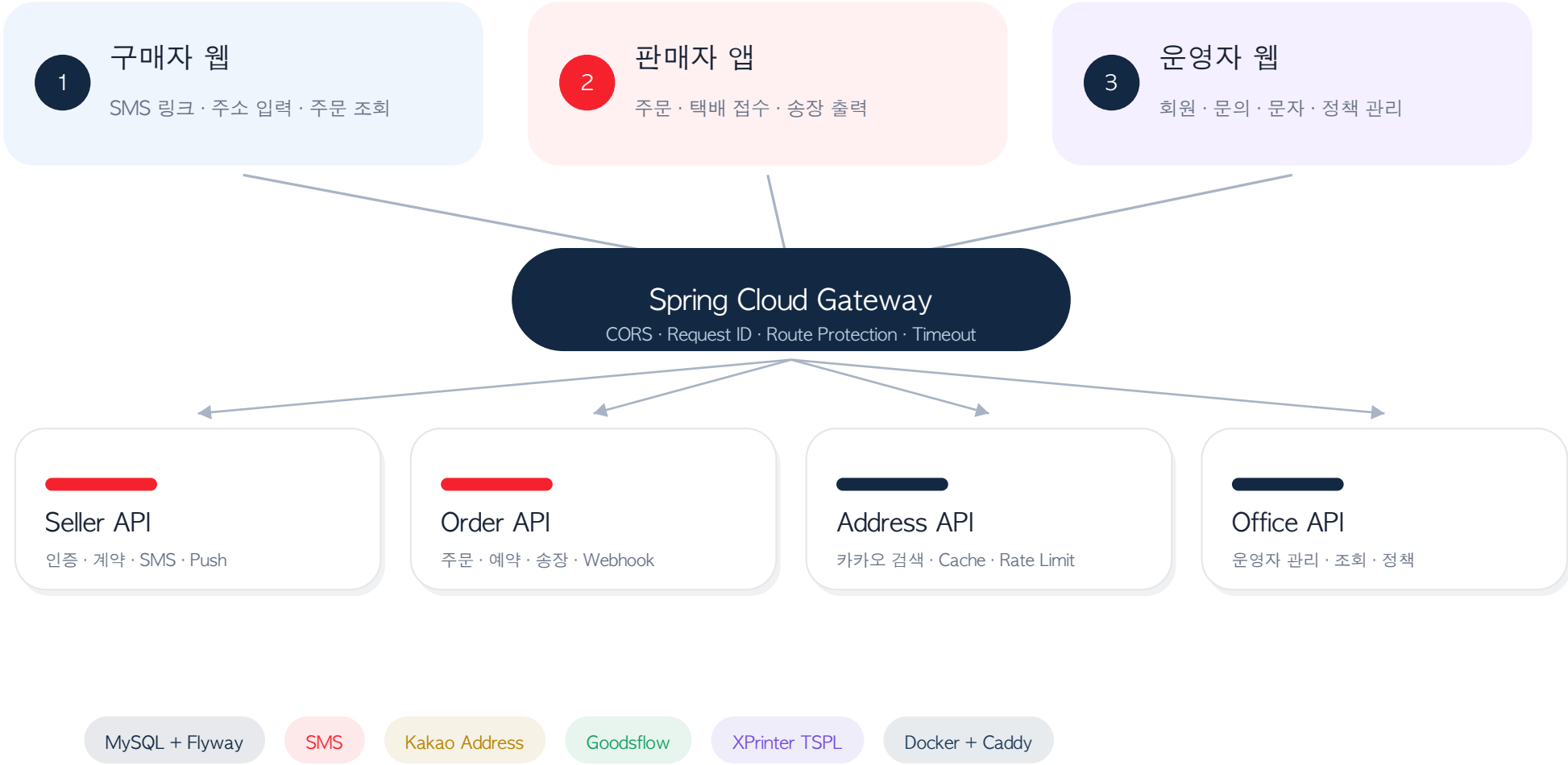
AFTER

주문왔슈에서 한 번에

- 문자 링크로 고객이 주소를 직접 입력
- 앱에서 주문 확인과 택배 접수
- 블루투스 송장 출력과 배송 확인

3개 클라이언트와 4개 핵심 API를 연결

API Gateway를 단일 진입점으로 두고 사용자 역할과 도메인 책임을 분리했습니다.



MySQL + Flyway | SMS | Kakao Address | Goodsflow | XPrinter TSPL | Docker + Caddy

화면 구현에서 출시·운영까지

사용자 여정과 서비스 경계를 함께 설계하며 End-to-End로 개발했습니다.

구매자 웹

주문 링크, 본인 확인, 주소 등록·수정, 주문 내역, 개인정보 동의, SEO 프리렌더링

React · TypeScript · Vite

판매자 앱

온보딩, 주소록, 주문, 택배 계약·요율, 알림, 접수·취소, 강제 업데이트

React Native · Expo

프린터 네이티브

Android·iOS XPrinter 연결, TSPL 전송, 상태 조회, 다중 송장 순차 출력

Kotlin · Swift · Objective-C

판매자·주문 API

인증, SMS·Push, 주문 생명주기, 택배 예약·취소, 송장 PDF, 배송 Webhook

Spring Boot · JPA

주소·운영 API

카카오 주소 검색, Cache·Rate Limit, 회원·문의·문자·운송장·버전 정책 관리

Spring Boot · MyBatis

배포·운영

Gateway, HTTPS, Docker 배포, Flyway, Request ID, Health Check, 실패 롤백

Gateway · Docker · Caddy

담당 역할

서비스 구조 설계 · 구매자 웹 · 판매자 앱 · 백엔드 API · 배포·운영 전반 주도

운영자 웹 일부 협업 | 주요 기능 및 기술 구조 주도

주문 전 과정을 하나의 사용자 여정으로

화면별 기능이 아니라 주문 생성부터 배송 처리까지 이어지는 전체 흐름을 기준으로 설계했습니다.

PROBLEM

주소 확인과 반복 입력으로 정보가 여러 곳에 분산

- 전화·문자로 주소를 다시 확인
- 판매자와 택배기사가 반복 입력
- 주문·접수·배송 상태가 분리

SOLUTION FLOW

링크 전송부터 배송 처리까지 연결

1 링크 전송

고객 전화번호로 일회성 주문 링크 발송

2 주소 입력

본인 확인 후 고객이 배송지를 직접 입력

3 주소 재사용

최근 배송지 불러오기·수정·신규 등록

4 택배 처리

접수·취소·배송 상태를 동일 흐름으로 관리

RESULT

주소 요청 → 주문 등록 → 택배 접수 → 배송 확인을 하나의 서비스 흐름으로 통합하고 실제 출시했습니다.

일회성 토큰 | 상태 정책 | 개인정보 동의

실물 프린터의 예외를 앱 내부에서 흡수

React Native 화면 연결을 넘어 Android·iOS 네이티브 브리지와 TSPL 통신 흐름을 직접 구현했습니다.

REAL DEVICE PROBLEM

OS와 프린터 모델마다 연결·응답 시점이 달랐습니다.

- 구형 Android의 늦거나 중복된 연결 콜백
- 다중 송장 전송 시 순서와 완료 시점 차이
- iOS BLE와 Android SDK의 서로 다른 인터페이스
- 연결 실패·용지 상태를 사용자에게 설명할 필요



구현한 안정화 장치

1. **세션 제어**
   중복 연결 차단, connection generation으로 오래된 콜백 무시
2. **순차 출력**
   TSPL Base64 다중 페이지를 배치로 전달하고 전송 순서 보장
3. **상태·타임아웃**
   프린터 상태 조회와 응답 제한 시간, 재연결·복구 처리
4. **안전 검증**
   위험한 명령과 과도한 payload를 전송 전에 차단
5. **진단 가능성**
   네이티브 전송 로그와 사용자용 오류 안내를 함께 구성

기술적 의미 웹 기술의 한계를 네이티브 코드와 프로토콜 수준에서 해결

기능 개발 이후의 운영까지 설계

배포 실패, 외부 API 지연, 개인정보, 장애 추적을 실제 운영 시나리오로 다뤘습니다.

1 Prebuild
전체 API 이미지 선빌드

2 Migrate
Flyway 1회 실행

3 Replace
서비스 순차 교체

4 Verify
Health Check

5 Rollback
실패 시 기존 이미지 복구

추적 가능성

- Gateway가 Request ID 생성·전파
- 서비스 로그에서 동일 요청 추적
- 30일 로그 보관과 파일 롤링

안전한 데이터

- Flyway 버전 기반 DB 변경
- PII 암호화 검증과 보관 기한
- 로그인·SMS·주소 요청 Rate Limit

외부 연동 복구

- Outbox 기반 알림 재처리
- 택배 연동 실패 복구 스케줄러
- Webhook 중복·상태 변경 대응

FULL-STACK PRODUCT OWNERSHIP

웹·앱·백엔드·운영을 연결한 개발 경험

하나의 기능보다 전체 사용자 여정과 운영 가능성을 기준으로 만들었습니다.

End-to-End

구매자부터 판매자·기사·운영자까지

Native

Kotlin·Swift·Objective-C 프린터 연동

Backend

도메인 분리·상태 정책·외부 연동 복구

Operations

배포·마이그레이션·추적·롤백

실제 서비스 확인

App Store 출시와 운영 중인 구매자 웹을 확인할 수 있습니다.

주문왔슈 App Store

apps.apple.com/kr/app/id6827154136

구매자 웹

customer-web.orderwatsyu.com

지원 포지션

풀스택 개발자

Web · Mobile · Backend · Infra